EXPERIMENT 3A – CLASSIFICATION MODELS

To train and compare Logistic Regression, Decision Tree and Random Forest models for predicting customer churn.

ALGORITHM

1. Load the processed CSV and separate X and y.
2. Split data into 80% training and 20% testing sets.
3. Train three classifiers and predict test labels.
4. Evaluate confusion matrix, accuracy, precision, recall and F1-score.
5. Plot metric comparison and identify the best model using F1-score.

In [ ]

Copy

```
import pandas as pd
import matplotlib.pyplot as plt
import os
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import *

class="cm"># Ensure dataset is available
raw_filename = class="s">"Telco-Customer-Churn.csv.csv"
preprocessed_filename = class="s">"CustomerChurn_Preprocessed.csv"

if not os.path.exists(preprocessed_filename) and not os.path.exists(raw_filenam
    print(class="s">"Downloading dataset...")
    url = class="s">"https://raw.githubusercontent.com/IBM/telco-customer-churn
    try:
        df_temp = pd.read_csv(url)
        df_temp.to_csv(raw_filename, index=False)
```

```
        class="cm"># Fallback dummy data generation if internet is disconnected
        import numpy as np
        dummy_data = pd.DataFrame({
            class="s">'gender': np.random.choice([class="s">'Male', class="s">'
            class="s">'SeniorCitizen': np.random.choice([0, 1], 100),
            class="s">'Partner': np.random.choice([class="s">'Yes', class="s">'
            class="s">'Dependents': np.random.choice([class="s">'Yes', class="s
            class="s">'tenure': np.random.randint(1, 72, 100),
            class="s">'PhoneService': np.random.choice([class="s">'Yes', class=
            class="s">'MultipleLines': np.random.choice([class="s">'No', class=
            class="s">'InternetService': np.random.choice([class="s">'DSL', cla
            class="s">'OnlineSecurity': np.random.choice([class="s">'Yes', clas
            class="s">'OnlineBackup': np.random.choice([class="s">'Yes', class=
            class="s">'DeviceProtection': np.random.choice([class="s">'Yes', cl
            class="s">'TechSupport': np.random.choice([class="s">'Yes', class="
            class="s">'StreamingTV': np.random.choice([class="s">'Yes', class="
            class="s">'StreamingMovies': np.random.choice([class="s">'Yes', cla
            class="s">'Contract': np.random.choice([class="s">'Month-to-month',
            class="s">'PaperlessBilling': np.random.choice([class="s">'Yes', cl
            class="s">'PaymentMethod': np.random.choice([class="s">'Electronic
            class="s">'MonthlyCharges': np.random.uniform(18.25, 118.75, 100),
            class="s">'TotalCharges': np.random.uniform(18.25, 8000.00, 100),
            class="s">'Churn': np.random.choice([class="s">'No', class="s">'Yes
        })
        dummy_data.to_csv(raw_filename, index=False)

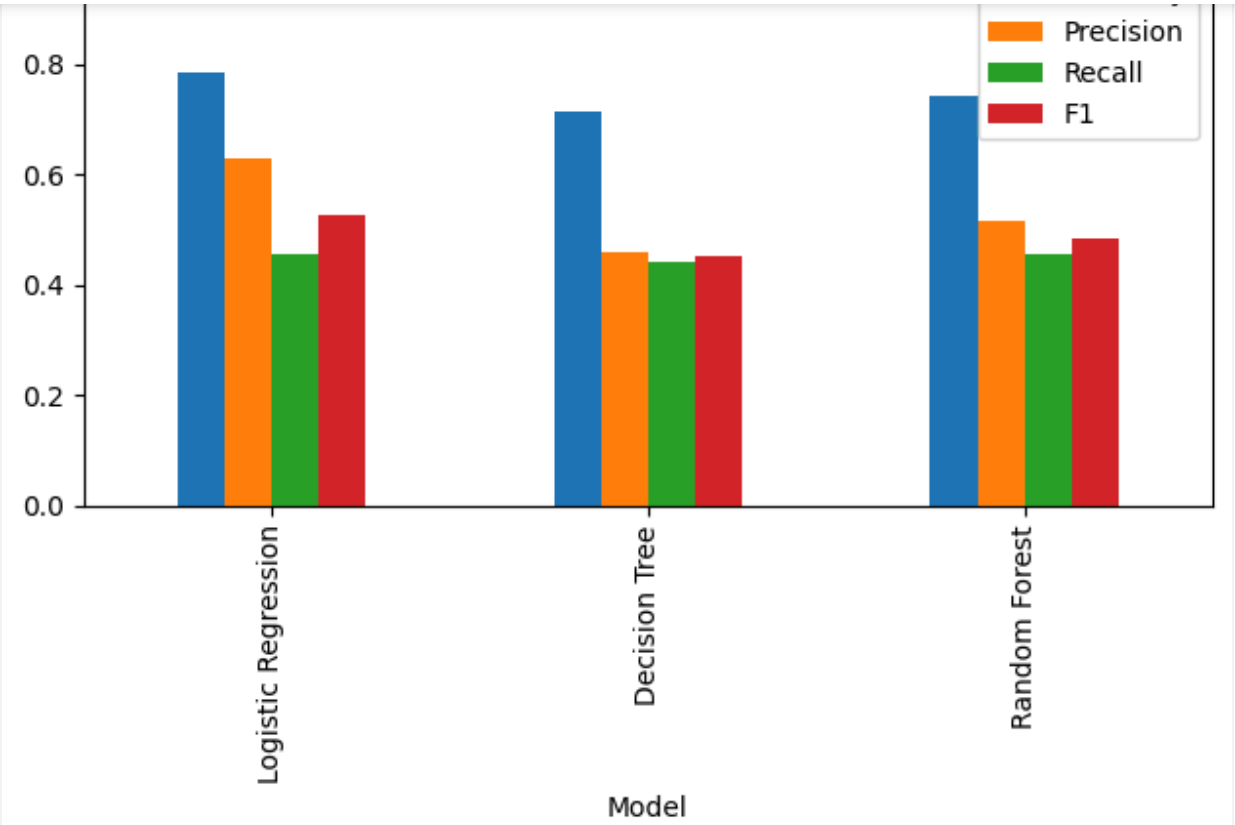
class="cm"># Load preprocessed dataset from Experiment 2
try:
    df = pd.read_csv(preprocessed_filename)
    y = df[class="s">"Churn"]
    X = df.drop(columns=[class="s">"Churn"], errors=class="s">"ignore")
except FileNotFoundError:
    class="cm"># Fallback to the raw CSV
    df = pd.read_csv(raw_filename)
    y = df[class="s">"Churn"].map({class="s">"No": 0, class="s">"Yes": 1})
    X = df.drop(columns=[class="s">"Churn",class="s">"customerID"], errors=clas

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=.2, random_state=42, stra
```

```
    class="s">"Random Forest": RandomForestClassifier(n_estimators=100, random_
}
rows = []
for name, m in models.items():
    m.fit(Xtr, ytr); pred = m.predict(Xte)
    print(name, class="s">"\n", confusion_matrix(yte, pred))
    rows.append([
        name,
        accuracy_score(yte, pred),
        precision_score(yte, pred, zero_division=0),
        recall_score(yte, pred, zero_division=0),
        f1_score(yte, pred, zero_division=0)
    ])

s = pd.DataFrame(rows, columns=[class="s">"Model", class="s">"Accuracy", class=
s.set_index(class="s">"Model").plot(kind=class="s">"bar", ylim=(0, 1)); plt.tig
print(class="s">"Best by F1:", s.loc[s.F1.idxmax(), class="s">"Model"])
```

```
Downloading dataset...
Successfully downloaded and saved Telco-Customer-Churn.csv.csv
Logistic Regression
 [[935 100]
 [204 170]]
Decision Tree
 [[842 193]
 [209 165]]
Random Forest
 [[875 160]
 [203 171]]
                 Model  Accuracy  Precision    Recall        F1
0  Logistic Regression  0.784244   0.629630  0.454545  0.527950
1        Decision Tree  0.714691   0.460894  0.441176  0.450820
2        Random Forest  0.742370   0.516616  0.457219  0.485106
```




```
Best by F1: Logistic Regression
```

RESULT

Thus, three classifiers are evaluated and the strongest churn prediction model is identified from the computed metrics.